



Warnsdorff 's Algorithm

Definition

- Warnsdorff's rule is a heuristic for finding a single knight's tour.
- The knight is moved so that it always proceeds to the square from which the knight will have the fewest onward moves.
- When calculating the number of onward moves for each candidate square, we do not count moves that revisit any square already visited.
- Example :

0	59	38	33	30	17	8	63
37	34	31	60	9	62	29	16
58	1	36	39	32	27	18	7
35	48	41	26	61	10	15	28
42	57	2	49	40	23	6	19
47	50	45	54	25	20	11	14
56	43	52	3	22	13	24	5
51	46	55	44	53	4	21	12

Warnsdorff's Algorithm –Knight's Tour Problem

```
import
java.util.concurrent.ThreadLocalRandom;

class GFG{

    public static final int N = 8;

    public static final int cx[] = {1, 1, 2, 2, -1, -1, -2, -2};

    public static final int cy[] = {2, -2, 1, -1, 2, -2, 1, -1};

    boolean limits(int x, int y) {

        return ((x >= 0 && y >= 0) &&

            (x < N && y < N));

    }

    boolean isempty(int a[], int x, int y) {

        return (limits(x, y)) && (a[y * N + x] < 0);

    }

}
```

```
int getDegree(int a[], int x, int y) {

    int count = 0;

    for (int i = 0; i < N; ++i)

        if (isempty(a, (x + cx[i]),

            (y + cy[i])))

            count++;

    return count; }

Cell nextMove(int a[], Cell cell) {

    int min_deg_idx = -1, c,

        min_deg = (N + 1), nx, ny;

    int start = ThreadLocalRandom.current().nextInt(1000) % N;

    for (int count = 0; count < N; ++count) {

}
```

Warnsdorff's Algorithm –Knight's Tour Problem

```
1.  int i = (start + count) % N;
2.      nx = cell.x + cx[i];
3.      ny = cell.y + cy[i];
4.      if ((isempty(a, nx, ny)) &&
5.          (c = getDegree(a, nx, ny)) < min_deg)
6.      {
7.          min_deg_idx = i;
8.          min_deg = c;      }  }
9.  if (min_deg_idx == -1)
10.     return null;

11.     nx = cell.x + cx[min_deg_idx];
12.     ny = cell.y + cy[min_deg_idx];

13.     a[ny * N + nx] = a[(cell.y) * N +
14.         (cell.x)] + 1;

15.     cell.x = nx;
16.     cell.y = ny;
17.     return cell; }
```

```
1.  void print(int a[]) {
2.      for (int i = 0; i < N; ++i) {
3.          for (int j = 0; j < N; ++j)
4.              System.out.printf("%d\t", a[j * N + i]);
5.              System.out.printf("\n");      }  }
6.  boolean neighbour(int x, int y, int xx, int yy) {
7.      for (int i = 0; i < N; ++i)
8.          if (((x + cx[i]) == xx) &&
9.              ((y + cy[i]) == yy))
10.             return true;
11.     return false; }
12. boolean findClosedTour() {
13.     // Filling up the chessboard matrix with -1's
14.     int a[] = new int[N * N];
15.     for (int i = 0; i < N * N; ++i)
16.         a[i] = -1;
```

```

1.  int sx = 3;
2.      int sy = 2;
3.  Cell cell = new Cell(sx, sy);
4.      a[cell.y * N + cell.x] = 1; // Mark first
    move.
5.  Cell ret = null;
6.      for (int i = 0; i < N * N - 1; ++i)  {
7.          ret = nextMove(a, cell);
8.          if (ret == null)
9.              return false;      } if (!neighbour(ret.x,
    ret.y, sx, sy))
10.         return false;
11.     print(a);
12.     return true;  }
13. public static void main(String[] args) {
14.

```

```

        while (!new GFG().findClosedTour())    {
            ;
        } } }
class Cell{
    int x;
    int y
    public Cell(int x, int y){
        this.x = x;
        this.y = y;    }
}

```

Output:

```

59  14  63  32  1  16  19  34
62  31  60  15  56  33  2  17
13  58  55  64  49  18  35  20
30  61  42  57  54  51  40  3
43  12  53  50  41  48  21  36
26  29  44  47  52  39  4  7
11  46  27  24  9  6  37  22
28  25  10  45  38  23  8  5

```



THANK YOU